KAFKA

1. WHAT PROBLEM KAFKA SOLVES

In traditional systems, applications directly interact with multiple downstream systems:

```
Order Service → Database
              → Analytics
              → Notifications
```

Problems:

- Tight coupling between systems
- If one system fails, it impacts others
- No buffering mechanism
- No replay capability
- Difficult to scale
- Mostly batch-based processing

Event-Based Architecture

Instead of directly calling systems, applications generate **events**.

Example:

```
Order Created → Event
```

These events are sent to a central system.

Kafka's Role

Kafka acts as:

- A **central event streaming platform**
- A **buffer between systems**
- A **durable storage system**
- A **real-time data pipeline backbone**

Flow:

```
Producer → Kafka → Multiple Consumers
```

2. KAFKA CORE CONCEPTS

Topic

A **topic** is a logical stream of data.

Examples:

```
orders
payments
clickstream
```

It is similar to:

- a table (in databases)
- a folder (in storage systems)

Partition

Each topic is divided into **partitions**.

```
orders topic:
   Partition 0
   Partition 1
   Partition 2
```

Why partitions exist:

- Enable parallel processing
- Enable horizontal scaling
- Improve throughput

Partition Internals

Each partition is an **append-only log**:

```
Partition 0:
0 → order1
1 → order2
2 → order3
```

Offset

- Offset is the **position of a message inside a partition**
- It is unique within a partition

Important:

- Offsets are **not global**
- Each partition maintains its own offsets

Producer

A **producer** sends data to Kafka.

Examples:

- Order service
- Payment service
- Web application

Responsibilities:

- Choose topic
- Decide partition
- Send data

Consumer

A **consumer** reads data from Kafka.

Examples:

- Spark job
- Analytics system
- Notification service

Important behavior:

- Kafka is **pull-based**
- Consumers decide:
    - when to read
    - how much to read

Consumer Group

A **consumer group** is a set of consumers working together.

Example:

```
Group G1:
   C1   C2   C3
```

Kafka distributes partitions among consumers.

Key Rule

- One partition is consumed by only **one consumer in a group**

Example

```
Partitions: P0 P1 P2
Consumers:  C1 C2
```

Assignment:

```
C1 → P0, P1
C2 → P2
```

Ordering Guarantee

- Ordering is guaranteed **only within a partition**
- Not across partitions

3. KAFKA ARCHITECTURE

Components

- **Broker** → Kafka server

- **Cluster** → group of brokers
- **Topic → Partitions → Logs**

Data Flow

```
Producer → Broker (Leader) → Followers → Consumer
```

Leader & Followers

Each partition has:

- 1 Leader → handles reads/writes
- Multiple Followers → replicate data

ISR (In-Sync Replicas)

- Set of replicas that are up-to-date
- Used for fault tolerance

4. PRODUCER INTERNLS

Partition Selection

```
If key present:
   partition = hash(key) % partitions
Else:
   round-robin
```

Batching

Producer does not send one message at a time.

It batches messages:

- `batch.size`
- `linger.ms`

Benefits:

- High throughput
- Efficient network usage

Acknowledgments (acks)

Value	Meaning
0	No guarantee
1	Leader only
all	Leader + replicas

Retries

Producer retries sending messages on failure.

Idempotence

`enable.idempotence=true`

Prevents duplicate writes.

5. CONSUMER INTERNALS

Consumer Workflow

1. Connect to broker
2. Join consumer group
3. Get partition assignment
4. Read messages
5. Commit offsets

Offset Management

Offsets are stored in Kafka internally.

Auto Commit

- Offsets committed automatically
- Risk: data loss

Manual Commit

- Controlled commit
- Safer but can cause duplicates

6. CONSUMER GROUP REBALANCING

What triggers rebalance?

- New consumer joins
- Consumer leaves
- Topic partition changes

What happens during rebalance?

1. Stop consumption
2. Reassign partitions
3. Resume

Impact

- Temporary downtime
- Possible duplicates

7. STORAGE INTERNLS

Log Structure

Kafka stores data in:

```
.log files
.index files
.timeindex files
```

Segment Files

Data is split into segments:

```
segment.bytes = 1GB
```

Why Kafka is Fast

- Sequential disk writes
- OS page cache
- Zero-copy transfer
- Batching

8. RETENTION

Kafka does NOT delete data after consumption.

Retention is based on:

- Time (`retention.ms`)
- Size (`retention.bytes`)

Example

```
retention.ms = 1 day
```

Data older than 1 day is deleted.

9. DELIVERY GUARANTEES

Type	Description
At most once	possible data loss
At least once	possible duplicates
Exactly once	no loss, no duplicates

Practical Usage

- Most systems use **at least once**
- Exactly once requires:
  - idempotence
  - transactions

10. FAULT TOLERANCE

Replication

Each partition is replicated across brokers.

Leader Failure

- New leader is elected from ISR
- No data loss if ISR is healthy

Configurations

- replication.factor
- min.insync.replicas

PERFORMANCE TUNING

1. PRODUCER TUNING (WRITE PATH OPTIMIZATION)

1.1 batch.size

What it is:

- Size of batch (in bytes) before sending to broker

Default:

```
16384 bytes (16 KB)
```

How it works:

Instead of:

```
Send message1
Send message2
Send message3
```

Kafka does:

```
Batch → [msg1, msg2, msg3] → send together
```

Impact:

Small batch	Large batch
Low latency	High throughput
More network calls	Fewer network calls

When to increase:

- High throughput pipelines
- Streaming logs
- Bulk ingestion

Risk:

- Increased latency

1.2 linger.ms

What it is:

- Time producer waits before sending batch

Example:

```
linger.ms = 0 → send immediately
linger.ms = 100 → wait 100ms to accumulate batch
```

Trade-off:

Low linger	High linger
Low latency	Better batching
Lower throughput	Higher throughput

Real-world use:

- Real-time alerts → low linger
- Data pipelines → higher linger

1.3 compression.type

Options:

```
none
gzip
snappy
lz4
zstd
```

Why compression matters:

- Reduces network usage
- Improves throughput

Trade-off:

Compression	CPU	Network
none	low	high
gzip	high	low
snappy	medium	good balance

1.4 acks (VERY IMPORTANT)

Value	Behavior
0	fastest, no guarantee
1	leader only
all	safest

Production standard:

```
acks=all
```

1.5 retries

- Retry failed sends
- Prevent transient failures

1.6 enable.idempotence

```
true → no duplicates from producer
```

PRODUCER SUMMARY

- Throughput = batch.size + linger.ms

- Reliability = acks + retries + idempotence

2. BROKER TUNING (CLUSTER LEVEL)

2.1 partitions

Most important scaling parameter

Rule:

```
More partitions → more parallelism
```

But:

Too many partitions →

- Metadata overhead
- Memory usage
- Rebalancing cost

Practical guideline:

- Start small (2–6)
- Scale based on throughput

2.2 replication.factor

```
1 → no fault tolerance
3 → production standard
```

Trade-off:

Low replication	High replication
fast	durable
risky	safe

2.3 min.insync.replicas

Works with:

```
acks=all
```

Example:

```
replication.factor=3
min.insync.replicas=2
```

At least 2 brokers must confirm write

2.4 retention

```
retention.ms
retention.bytes
```

Important understanding:

Kafka = log system
NOT queue

2.5 segment.bytes

- Size of log segment

Smaller segments:

- Faster cleanup
- More files

3. CONSUMER TUNING (READ PATH)

3.1 fetch.min.bytes

Minimum data to fetch

Behavior:

- Wait until enough data accumulates

3.2 max.poll.records

Max messages per poll

Impact:

- Controls processing batch size

3.3 parallelism

Controlled by:

```
number of partitions
```

3.4 auto.offset.reset

```
earliest → read from beginning
latest → read new data only
```

3.5 enable.auto.commit

```
true → automatic
false → manual control
```

CONSUMER SUMMARY

- Performance = fetch + poll tuning
- Reliability = offset control

COMPLETE KAFKA LAB

LAB GOAL

You will demonstrate:

COMPLETE KAFKA LAB — KRaft EDITION

Updated to match **docker-compose.kafka.yml** (Apache Kafka 3.9.1, single-node KRaft, no ZooKeeper). The preceding Kafka concepts and tuning lessons remain unchanged.

Goal: Producer → Kafka → Consumer, with a parallel PySpark Structured Streaming consumer in Jupyter. All commands below assume Docker Compose v2.

STEP 0 — PREPARE AND START

Save **docker-compose.kafka.yml** in the same folder as the existing lab **docker-compose.yml**. The standalone file uses the existing external Docker network **labnet**. Do not start the old Kafka and ZooKeeper services alongside it.

```
docker compose -f docker-compose.yml stop kafka zookeeper
docker compose -f docker-compose.yml rm -f kafka zookeeper
# If labnet does not already exist:
docker network inspect labnet >/dev/null 2>&1 || docker network create labnet
docker compose -f docker-compose.kafka.yml up -d
docker compose -f docker-compose.kafka.yml ps
```

Keep Jupyter running from the main lab if you need Spark streaming. Start it with **docker compose -f docker-compose.yml up -d jupyter** if necessary. Do not run **docker compose up -d** without specifying services, because that would also start the old Kafka stack.

STEP 1 — VERIFY KAFKA IS HEALTHY

```
docker ps --filter name=kafka-standalone
docker logs kafka-standalone --tail 80
docker inspect kafka-standalone --format="{{.State.Health.Status}}"
```

Expected: Kafka container is running and eventually reports **healthy**. The image contains Kafka CLI tools under **/opt/kafka/bin/**. KRaft uses an internal controller and does not require ZooKeeper.

STEP 2 — ENTER THE KAFKA CONTAINER

```
docker exec -it kafka-standalone bash
```

For Steps 3–8, run commands **inside** the Kafka container. The broker address inside this container is **localhost:19092**.

STEP 3 — CREATE THE ORDERS TOPIC

```
/opt/kafka/bin/kafka-topics.sh --create \
  --topic orders \
  --bootstrap-server localhost:19092 \
  --partitions 2 \
  --replication-factor 1
```

Two partitions allow parallel consumption. Replication factor 1 is appropriate for this one-broker teaching lab but does not provide broker-level fault tolerance. If the topic already exists, skip creation.

STEP 4 — VERIFY TOPIC AND PARTITIONS

```
/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:19092
/opt/kafka/bin/kafka-topics.sh --describe \
  --topic orders --bootstrap-server localhost:19092
```

Discuss partition count, leader, replicas, and ISR. On this single-node lab, the same broker leads both partitions.

STEP 5 — START A LIVE CONSUMER (TERMINAL 1)

Open a **new host terminal** and run:

```
docker exec -it kafka-standalone \
  /opt/kafka/bin/kafka-console-consumer.sh \
  --topic orders --bootstrap-server localhost:19092
```

The consumer waits for new messages. It may initially show no output.

STEP 6 — START A PRODUCER (TERMINAL 2)

```
docker exec -it kafka-standalone \
  /opt/kafka/bin/kafka-console-producer.sh \
  --topic orders --bootstrap-server localhost:19092
```

Type one message per line and press Enter:

```
order1,100
order2,200
order3,300
```

The consumer in Terminal 1 should display the new messages. This demonstrates live publish/subscribe behavior.

STEP 7 — EXPLORE CONSUMER GROUPS

Stop the first consumer with Ctrl+C. Start a group consumer, then send a few **new** messages from the producer. A new group without an explicit reset policy normally begins at the latest available position.

```
docker exec -it kafka-standalone \
  /opt/kafka/bin/kafka-console-consumer.sh \
  --topic orders --group g1 --bootstrap-server localhost:19092
```

Optional: start a second consumer with **--group g1** in another terminal. Kafka assigns the two topic partitions across group members. Message ordering is guaranteed within each partition, not globally.

STEP 8 — CHECK OFFSETS AND LAG

```
docker exec kafka-standalone \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --describe --group g1 --bootstrap-server localhost:19092
```

CURRENT-OFFSET is the next offset committed for the group, **LOG-END-OFFSET** is the partition end, and **LAG** is their difference. Offset commits may not appear immediately; wait briefly and rerun if necessary.

STEP 9 — PYSPARK STRUCTURED STREAMING IN JUPYTER

Open **http://localhost:8888** and use the existing lab token **training123**. Your Jupyter container and Kafka must both be attached to **labnet**. In Jupyter, the Kafka bootstrap address is **kafka:19092**, not localhost.

Important: Spark requires the matching **spark-sql-kafka-0-10** connector. Its version must match the Spark version installed in Jupyter. The existing **jupyter/pyspark-notebook:latest** image does not pin that version. Run the following in a fresh notebook before creating a SparkSession:

```
import pyspark
print("PySpark version:", pyspark.__version__)
```

Set **SPARK_VERSION** to the exact printed version (for example, 3.5.3 if that is what your image reports). A working network connection to Maven repositories is required if the connector is not already cached.

```
import pyspark
from pyspark.sql import SparkSession

SPARK_VERSION = pyspark.__version__
spark = (SparkSession.builder
    .appName("KafkaLab")
    .config("spark.jars.packages",
        f"org.apache.spark:spark-sql-kafka-0-10_2.12:{SPARK_VERSION}")
    .getOrCreate())
```

The example uses Scala 2.12 artifacts, appropriate for common Spark 3.x builds. If your Jupyter image contains Spark 4.x or a Scala 2.13 build, use the matching connector artifact and Scala binary version instead. Restart the kernel after changing connector settings.

```
df = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "kafka:19092")
    .option("subscribe", "orders")
    .option("startingOffsets", "latest")
    .load())

query = (df.selectExpr("CAST(value AS STRING) AS order_event")
    .writeStream
    .format("console")
    .outputMode("append")
    .option("checkpointLocation", "/tmp/kafka-lab-checkpoint")
    .start())
print("Streaming active:", query.isActive)
```

Send new messages while the query runs. Console sink output appears in the **Jupyter container logs**, not necessarily as notebook cell output:

```
docker logs -f jupyter
```

To stop the query in the notebook, run **query.stop()**. To replay existing records in a fresh query, change startingOffsets to **earliest** and use a new checkpoint location; an existing checkpoint takes precedence.

STEP 10 — FINAL LIVE TEST

With the producer, group consumer, and Spark stream running, send:

```
order10,500
order11,700
```

Verify the consumer prints the events, and check Jupyter logs for Spark micro-batch output. A consumer group and Spark streaming query are independent readers; both can read the same Kafka events.

TROUBLESHOOTING — IF KAFKA STOPS

```
docker logs kafka-standalone --tail 150
docker inspect kafka-standalone \
  --format="ExitCode={{.State.ExitCode}} OOMKilled={{.State.OOMKilled}}"
docker stats --no-stream
docker network inspect labnet
```

If **OOMKilled=true**, reduce other lab workloads or increase Docker memory. If Jupyter cannot connect, verify both containers are on **labnet** and use **kafka:19092**. If Spark reports **Failed to find data source: kafka**, correct the Spark Kafka connector version. If port 9092 is already in use, ensure the old Kafka container is stopped.

Do not use docker compose down -v unless you intentionally want to delete persistent Kafka topic data. For normal shutdown: **docker compose -f docker-compose.kafka.yml down**.

CONNECTION REFERENCE

Host Python: localhost:9092 | **Other lab containers:** kafka:19092 | **Kafka container CLI:** localhost:19092 | **Controller:** kafka:9093 (internal only).

Final flow: Producer → Kafka orders topic → Console consumer / Consumer group / Spark Structured Streaming.